

LAPORAN TUGAS II
SISTEM OPERASI (PROCESS)



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Di susun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2024

1. TUJUAN

1.1. Process Concept dan Process Operation

1. Memahami konsep proses dan siklus hidupnya
2. Mampu membuat dan mengelola proses menggunakan system call
3. Memahami hubungan parent-child process
4. Menganalisis Process ID (PID) dan Process Control Block.

1.2. Interprocess Communication (IPC)

1. Memahami konsep komunikasi antar proses
2. Mengimplementasikan Shared Memory IPC
3. Mengimplementasikan Message Passing dengan Pipes
4. Membandingkan performa kedua metode

2. DASAR TEORI

2.1. KONSEP PROCESS

Secara informal; proses adalah program dalam eksekusi. Suatu proses adalah lebih dari kode program, dimana kadang kala dikenal sebagai bagian tulisan. Proses juga termasuk aktivitas yang sedang terjadi, sebagaimana digambarkan oleh nilai pada program counter dan isi dari daftar prosesor/ processor's register. Suatu proses umumnya juga termasuk process stack, yang

berisikan data temporer (seperti parameter metoda, address yang kembali, dan variabel lokal) dan sebuah data section, yang berisikan variabel global. Kami tekankan bahwa program itu sendiri bukanlah sebuah proses; suatu program adalah satu entitas pasif; seperti isi dari sebuah berkas yang disimpan didalam disket, sebagaimana sebuah proses dalam suatu entitas aktif, dengan sebuah program counter yang mengkhususkan pada instruksi selanjutnya untuk dijalankan dan seperangkat sumber daya/ resource yang berkenaan dengannya.

Walau dua proses dapat dihubungkan dengan program yang sama, program tersebut dianggap dua urutan eksekusi yang berbeda. Sebagai contoh, beberapa pengguna dapat menjalankan copy yang berbeda pada mail program, atau pengguna yang sama dapat meminta banyak copy dari program editor. Tiap-tiap proses ini adakah proses yang berbeda dan walau bagian tulisan-text Adalah sama,

data section bervariasi. Juga adalah umum untuk memiliki proses yang menghasilkan banyak proses begitu ia bekerja.

3. PERCOBAAN

3.1. SISTEM MANAGEMENT PROCESS

3.1.1. Source Code

Membuat file `proses_manajemen.c`, Lalu Masukkan kode Seperti Di bawah.

```
tugas > C proses_menejemen.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6  #include <time.h>
7  #include <string.h>
8
9  // PEMBUATAN CHILD
10 // child 1 : SORTING
11 void sorting() {
12     printf(" [PID %d] mulai sorting.\n", getpid());
13     int arr[] = {5, 2, 8, 1, 9, 4, 7, 3, 6}; // array awal
14     int n = sizeof(arr) / sizeof(arr[0]);
15     int i, j, temp;
16
17
18     usleep(200000); // mengatur waktu eksekusi 2 detik
19     // BUBBLE SORT
20     for (i = 0; i < n - 1; i++) { // LOOPING
21         for (j = 0; j < n - i - 1; j++) { // LOOPING
22             if (arr[j] > arr[j + 1]) {
23                 temp = arr[j];
24                 arr[j] = arr[j + 1];
25                 arr[j + 1] = temp;
26             }
27         }
28         printf(" [PID %d] Selesai SORTING. hasil urut : %d, %d, ..., %d\n", getpid(), arr[0], arr[1], arr[n-1]);
29         exit(0); // Keluar dengan status sukses (0)
30     }
31
32 // child 2 : SEARCHING
33
34 void searching() {
35     printf(" [PID %d] Mulai searching.\n", getpid());
36     int arr[] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
37     int target = 60;
38     int found = -1;
39     int n = sizeof(arr) / sizeof(arr[0]);
40     usleep(150000); // mengatur waktu eksekusi 1,5 detik
41     for (int i = 0; i < n; i++) {
42         if (arr[i] == target) {
43             found = i;
44             break;
45         }
46     }
47     if (found != -1) {
48         printf(" [PID %d] Selesai SEARCHING. Ditemukan %d di indeks %d.\n", getpid(), target, found);
49     } else {
50         printf(" [PID %d] Selesai SEARCHING. Target %d tidak ditemukan.\n", getpid(), target);
51     }
52     exit(0); // Keluar dengan status sukses (0)
53 }
54 }
```

```

56 // Child 3 : CALCULATION
57
58 void calculation() {
59     printf(" [PID %d] Mulai tugas CALCULATION (Faktorial 7).\n", getpid());
60     long long hasil = 1;
61     int angka = 7;
62     // Simulasi waktu eksekusi
63     usleep(1600000); //mengatur waktu eksekusi 1,6 detik
64     for (int i = 2; i <= angka; i++) {
65         hasil *= i;
66     }
67
68     printf(" [PID %d] Selesai CALCULATION. hasil Faktorial %d = %lld\n", getpid(), angka, hasil);
69     exit(0); // Keluar dengan status sukses (0)
70 }
71
72 int main() {
73     pid_t pids[3];
74     int status;
75     clock_t waktu_mulai[3], wkt_selesai[3];
76     double waktu_eksekusi[3];
77     const char *tugas[] = {"SORTING", "SEARCHING", "CALCULATION"};
78     void (*arr_tugas[])() = {sorting, searching, calculation};
79     int jmlh_proses = 3;
80     printf("=== Sistem Manajemen Proses (Parent PID: %d) ===\n\n", getpid());
81     // 1. Membuat multiple child proses
82     for (int i = 0; i < jmlh_proses; i++) {
83         waktu_mulai[i] = clock(); // Catat waktu mulai
84         pids[i] = fork();
85
86         if (pids[i] < 0) {
87             // Error
88             perror("Gagal membuat proses anak (fork)");
89             return 1;
90         } else if (pids[i] == 0) {
91             arr_tugas[i](); // 2. Setiap process melakukan tugas berbeda
92         }
93     }
94     // 3. Parent process mengkoordinasi dan mengumpulkan hasil
95     usleep(3000000); // Tunda 1 detik
96
97     printf("\n[Parent PID %d] Semua proses anak telah diluncurkan. Menunggu proses anak selesai...\n\n", get
98
99     for (int i = 0; i < jmlh_proses; i++) {
100         pid_t terminated_pid = waitpid(pids[i], &status, 0);
101         end_time[i] = clock(); // Catat waktu selesai
102
103         if (terminated_pid == -1) {
104             perror("waitpid gagal");
105             continue;
106         }
107     }
108 }

```

```

// 4. Tampilkan PID, execution time, dan status setiap proses
// Hitung waktu eksekusi
waktu_eksekusi[i] = ((double)(wkt_selesai[i] - waktu_mulai[i])) / CLOCKS_PER_SEC;

printf("--- CHILD  %d ---\n", i + 1);
printf("  PID: %d\n", (int)pids[i]);
printf("  Tugas: %s\n", tugas[i]);
printf("  Waktu Eksekusi: %.3f detik\n", waktu_eksekusi[i]);

// Cek status
if (WIFEXITED(status)) {
    printf("  Status: SELESAI NORMAL (Exit Code: %d)\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("  Status: TERMINASI OLEH SINYAL (Sinyal: %d)\n", WTERMSIG(status));
} else {
    printf("  Status: STATUS TIDAK DIKETAHUI\n");
}
printf("-----\n");
}
printf("\n[Parent PID %d] Semua proses anak telah selesai. Program berakhir.\n", getpid());
return 0;
}

```

3.1.2. Algoritma

A. Alur Algoritma Program Utama (Parent)

1. Inisialisasi:

- Program utama dimulai dan mencetak PID (Process ID) dirinya sendiri.
- Deklarasi dan inisialisasi variabel untuk menyimpan PID proses anak (pids), status keluar (status), waktu eksekusi (waktu_mulai, wkt_selesai, waktu_eksekusi), nama tugas (tugas), dan fungsi tugas (arr_tugas).
- Variabel jmlh_proses diset ke 3.

2. Membuat Proses Child:

- Sebuah loop for dijalankan sebanyak tiga kali (sesuai jmlh_proses).
- Di setiap iterasi:
- Waktu saat ini dicatat sebagai waktu_mulai.
- Fungsi fork() dipanggil untuk membuat proses anak baru.

- Jika `fork()` berhasil (PID anak sama dengan 0), maka program melompat ke Alur Algoritma Proses Anak untuk menjalankan salah satu fungsi tugas (sorting, searching, atau calculation). Setelah selesai, proses anak akan keluar dengan `exit(0)`.
- Jika `fork()` gagal, program akan mencetak pesan kesalahan dan berakhir.
- Jika `fork()` berhasil (PID anak tidak sama dengan 0), maka program tetap berada di dalam proses utama (parent) dan melanjutkan ke iterasi loop berikutnya untuk membuat proses anak selanjutnya.

3. Menunggu Proses Anak:

- Setelah semua proses anak diluncurkan, program utama akan menunggu semua proses anak selesai menggunakan sebuah *loop* for dan fungsi `waitpid()`.
- Di setiap iterasi *loop*, `waitpid()` akan memblokir proses utama hingga salah satu proses anak dengan PID tertentu (`pids[i]`) selesai.
- Saat proses anak selesai, waktu penyelesaian (`wkt_selesai`) dicatat.

4. Mengumpulkan dan Menampilkan Hasil:

- Setelah setiap proses anak selesai dan `waitpid()` kembali, proses utama melakukan hal-hal berikut:
 - Menghitung waktu eksekusi proses anak dengan mengurangi waktu mulai dari waktu selesai.
 - Mencetak informasi detail dari proses anak yang baru saja selesai, termasuk PID, nama tugas, dan waktu eksekusinya.
 - Memeriksa status untuk menentukan apakah proses anak selesai secara normal atau terhenti oleh sinyal, lalu mencetak status tersebut.
 - Setelah semua proses anak selesai, sebuah pesan penutup akan dicetak.

5. Pengakhiran:

- Program utama berakhir dengan `return 0`.

B. Alur Algoritma Proses Child

- Ketika sebuah proses anak dibuat, ia akan menjalankan salah satu fungsi berikut, bergantung pada urutan `fork()`:

Alur Algoritma `sorting()` (Child 1)

1. Cetak pesan bahwa proses sorting dimulai, beserta PID-nya.
2. Inisialisasi array `arr` dengan nilai yang tidak terurut.
3. Tunda eksekusi selama 2 detik dengan `usleep()`.

4. Lakukan pengurutan array menggunakan algoritma **Bubble Sort**.
5. Cetak pesan bahwa proses sorting selesai, beserta PID dan hasil array yang sudah terurut.
6. Keluar dari proses dengan status sukses (exit(0)).

Alur Algoritma searching() (Child 2)

1. Cetak pesan bahwa proses searching dimulai, beserta PID-nya.
2. Inisialisasi array arr yang sudah terurut dan tentukan nilai target yang ingin dicari (60).
3. Tunda eksekusi selama 1.5 detik dengan usleep().
4. Lakukan pencarian nilai target dalam array.
5. Jika target ditemukan, cetak pesan bahwa pencarian selesai dan di indeks mana nilai tersebut ditemukan.
6. Jika target tidak ditemukan, cetak pesan bahwa pencarian selesai dan nilai tidak ditemukan.
7. Keluar dari proses dengan status sukses (exit(0)).

Alur Algoritma calculation() (Child 3)

1. Cetak pesan bahwa proses calculation dimulai (menghitung faktorial 7), beserta PID-nya.
2. Inisialisasi variabel hasil dan angka (7).
3. Tunda eksekusi selama 1.6 detik dengan usleep().
4. Hitung nilai faktorial dari angka dengan sebuah *loop* for.
5. Cetak pesan bahwa proses calculation selesai, beserta PID dan hasil faktorial.
6. Keluar dari proses dengan status sukses (exit(0)).

3.1.3. Output

Untuk mengCompile file proses_manajemen Tulis kode **gcc -o proses_menejemen proses_menejemen.c**
Jika tidak ada pesan error, langsung running dengan cara **./proses_menejemen**. Maka hasilnya seperti gambar dibawah.

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/tugas$ gcc -o proses_menejemen proses_menejemen.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/tugas$ ./proses_menejemen
=== Sistem Manajemen Proses (Parent PID: 882) ===

[PID 884] Mulai searching.
[PID 883] mulai sorting.
[PID 885] Mulai tugas CALCULATION (Faktorial 7).
[PID 883] Selesai SORTING. hasil urut : 1, 2, ..., 9
[PID 884] Selesai SEARCHING. Ditemukan 60 di indeks 5.
[PID 885] Selesai CALCULATION. hasil Faktorial 7 = 5040

[Parent PID 882] Semua proses anak telah diluncurkan. Menunggu proses anak selesai...
```

```

--- CHILD 1 ---
PID: 883
Tugas: SORTING
Waktu Eksekusi: 0.001 detik
Status: SELESAI NORMAL (Exit Code: 0)
-----

```

```

--- CHILD 2 ---
PID: 884
Tugas: SEARCHING
Waktu Eksekusi: 0.001 detik
Status: SELESAI NORMAL (Exit Code: 0)
-----

```

```

--- CHILD 3 ---
PID: 885
Tugas: CALCULATION
Waktu Eksekusi: 0.001 detik
Status: SELESAI NORMAL (Exit Code: 0)
-----

```

[Parent PID 882] Semua proses anak telah selesai. Program berakhir.
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/tugas\$ █

3.2.CHAT APPLICATION WITH IPC

Untuk mengCompile file proses_manajemen Tulis kode `gcc -o chat_client chat_client.c` dan `gcc -o chat_server chat_server.c` Jika tidak ada pesan error, lalu buka 3 terminal langsung running dengan cara `./chat_client <namauser>` untuk memulai obrolan Maka hasilnya seperti gambar dibawah.

```

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/tugas$ ./chat_client wiramatajaya
--- Aplikasi Chat Sederhana ---
User saat ini: wiramataj

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: 2

--- HISTORY PESAN ---
[wira]: haloo semuaa
[jaya]: halo wira
[wira]: bagaimana kabarmuuu!!!
[jaya]: baikkk, terimakasihii
-----

```

```

[CLIENT wira] Keluar.
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_2/tugas$ ./chat_client wira
--- Aplikasi Chat Sederhana ---
User saat ini: wira

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: 1
Masukkan pesan: haloo semuaa
[CHAT] Pesan terkirim: "haloo semuaa"

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: bagaimanaa kabarmuuu!!!
Pilihan tidak valid. Coba lagi.

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: 1

```

```

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: 1
Masukkan pesan: halo wira
[CHAT] Pesan terkirim: "halo wira"

Menu:
1. Kirim Pesan
2. Lihat History
3. Exit
Pilihan Anda: 1
Masukkan pesan: baikkk, terimakasihii
[CHAT] Pesan terkirim: "baikkk, terimakasihii"

```


4. KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. **Konsep Proses (Process Concept)** merupakan dasar dari pengelolaan eksekusi program dalam sistem operasi. Proses adalah program yang sedang dieksekusi, memiliki *state*, *program counter*, *register*, serta *memory space* tersendiri. Melalui praktikum ini, dipahami bahwa sistem operasi berperan penting dalam membuat, menjadwalkan, dan mengatur proses agar dapat berjalan secara efisien dan tidak saling mengganggu.
2. **Operasi pada Proses (Process Operations)** seperti *process creation*, *termination*, dan *scheduling* menjadi mekanisme utama dalam manajemen proses. Praktikum menunjukkan bagaimana proses induk (*parent process*) dapat membuat proses anak (*child process*) menggunakan sistem call seperti `fork()`, serta bagaimana sistem operasi menangani komunikasi dan penghapusan proses. Selain itu, pemahaman tentang *process state transition* (ready, running, waiting, terminated) menjadi penting untuk memahami siklus hidup proses.
3. **Komunikasi Antarproses (Interprocess Communication / IPC)** memungkinkan proses yang berbeda untuk saling bertukar data dan berkoordinasi. Melalui mekanisme seperti *pipe*, *message queue*, *shared memory*, dan *semaphore*, sistem operasi menyediakan cara agar proses dapat bekerja secara paralel dan sinkron tanpa konflik data. Praktikum ini memperlihatkan pentingnya sinkronisasi agar tidak terjadi *race condition* atau *deadlock* antarproses.
4. Secara keseluruhan, ketiga praktikum ini memperkuat pemahaman bahwa **manajemen proses dan komunikasi antarproses merupakan inti dari kinerja sistem operasi modern**, karena hampir semua aktivitas sistem bergantung pada efisiensi dan sinkronisasi proses-proses yang berjalan bersamaan.